

Modelos de Clasificación

Ciencia de datos para la toma de decisiones I

Jorge Juvenal
Campos
Ferreira

✉ juvenal.campos@tec.mx

¿Qué es clasificación?

★ Clasificación

Tarea **supervisada** en la que el algoritmo aprende a asignar cada observación a una **clase discreta** (etiqueta), a partir de un conjunto de predictores (**X**). La salida no es un numero continuo sino una categoria (**y**).

Ejemplos cotidianos

- **Correo:** spam / no spam
- **Banco:** aprobar / rechazar credito
- **Hospital:** diagnostico positivo / negativo
- **Educación:** desertó / continuó
- **Laboral:** Formalidad / informalidad

¿Qué aprende el modelo?

Aprende una **función** que **mapea predictores (X) a una probabilidad de pertenecer a cada clase**. Luego, una regla de decisión (*threshold de probabilidad*) convierte la probabilidad en la etiqueta final.

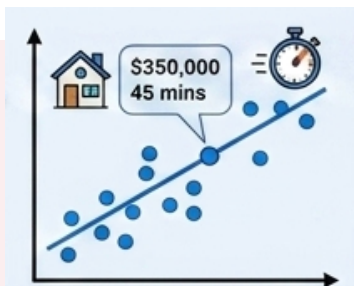
Idea clave: en clasificación no predecimos un numero, predecimos la **CATEGORIA** a la que pertenece una observación.

Regresión vs Clasificación

La pregunta cambia: en lugar de **cuanto**, preguntamos **cual**.

Regresión

Variable Y: numerica continua



Salida del modelo: un numero real

Ejemplos:

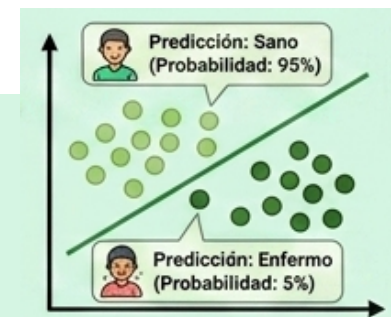
- Precio de una vivienda
- Tiempo de entrega en minutos
- Consumo en kWh

Métricas típicas:

RMSE (Raíz del error cuadrático medio), MAE (Mean Average Error), R cuadrada (R^2).

Clasificación

Variable Y: categorica (factor)



Salida del modelo: una clase + su probabilidad

Ejemplos:

- Aprobar / rechazar
- Desertó / no desertó
- Enfermo / sano

Métricas típicas:

Exactitud o *Accuracy*, Sensitividad, Especificidad, Área bajo la curva ROC

Tipos de problemas de clasificación

Binaria:

Es la forma más básica de clasificación, donde el modelo debe decidir entre exactamente **dos** **clases excluyentes**.

Ejemplos:

- spam / no spam
- desierto / no
- positivo / negativo

Modelo base:

Regresión logística

Multiclase:

Se utiliza cuando existen **3 o más** **clases posibles**, pero cada observación pertenece a una sola de ellas.

Ejemplos:

- **Iris**: setosa, versicolor, virginica
- Carrera elegida (entre 5)

Modelo base:

Regresión logística multinomial

Multietiqueta:

Permite que cada observación sea asignada a **varias categorías simultáneamente**.

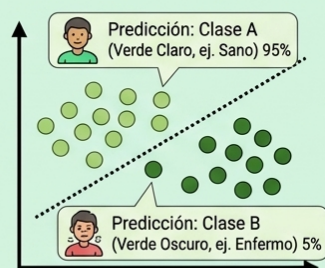
Ejemplos:

- Etiquetar fotos con varios tags
- Paciente con síntomas múltiples

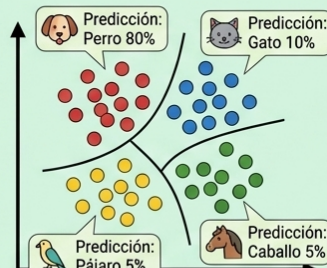
Modelo base:

Un clasificador binario por etiqueta

1. Clasificación Binaria (Binary)



2. Clasificación Multiclase (Multiclass)



3. Clasificación Multietiqueta (Multilabel)



Modelos de clasificación

Probabilísticos / lineales

- Regresión logística
- Análisis discriminante lineal (LDA) y cuadrático (QDA)
- Naive Bayes

Basados en distancia

- K-Nearest Neighbors (KNN)

Basados en márgenes

- Support Vector Machines (SVM)

Basados en árboles

- Árboles de decisión
- Random Forest
- Gradient Boosting

Redes neuronales

Métodos de ensamble (meta-modelos)

- Bagging, Boosting, Stacking, Voting

Otros que conviene tener en el radar

- Regresión logística penalizada (Lasso, Ridge, Elastic Net)
- Modelos basados en reglas (RIPPER, CN2)

En R, los paquetes de trabajo (los motores de los modelos) son **glmnet**, **ranger**, **xgboost**, **lightgbm**, **e1071** (SVM, Naive Bayes), **MASS** (LDA/QDA), **class** (**KNN**), todo unificable con **tidymodels**.

Regresión logística

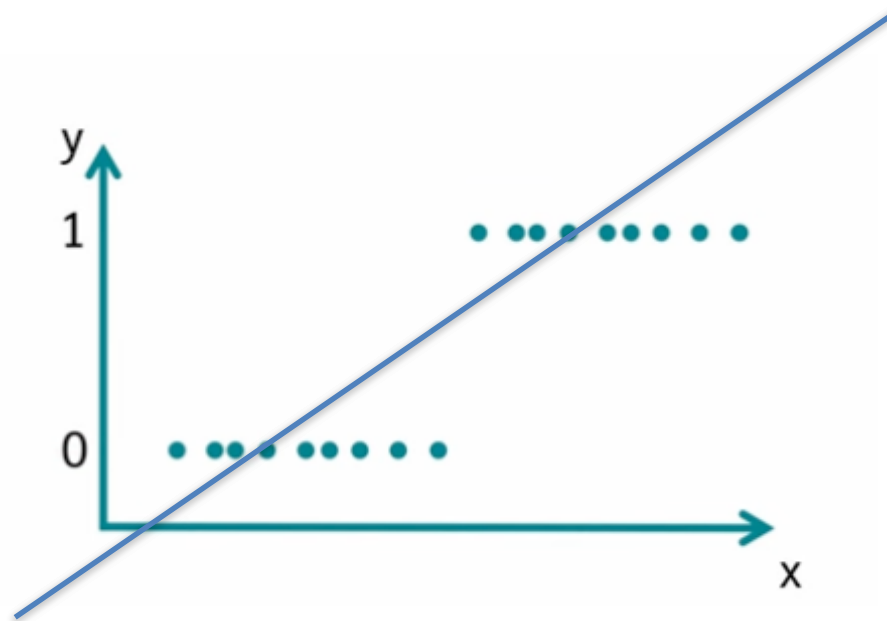
Es el análisis de clasificación **más básico** para predecir el resultado de una variable categórica en función de variables independientes o predictoras.

En el caso de una variable dicotómica (2 categorías) tenemos solo dos valores posibles para y : 0 o 1. Sea cual sea el valor de $\mathbf{X}$, el valor de $\mathbf{y}$ solo podrá ser 0 o 1.



Regresión logística

Si aplicásemos una regresión lineal a este problema, entonces, dados los valores de $\mathbf{X}$, esa regresión podría tomar valores distintos a 0 o 1. Podría ir de $-\infty$ a $+\infty$.

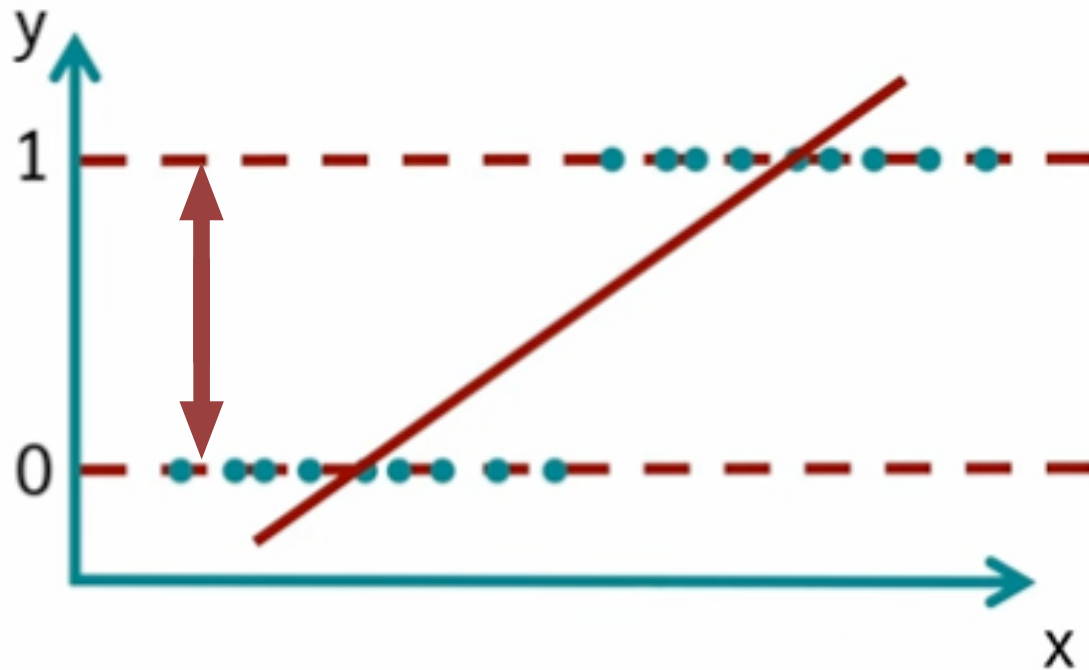


Regresión logística

El objetivo de la regresión logística es estimar la **probabilidad** de ocurrencia de 0 o 1.

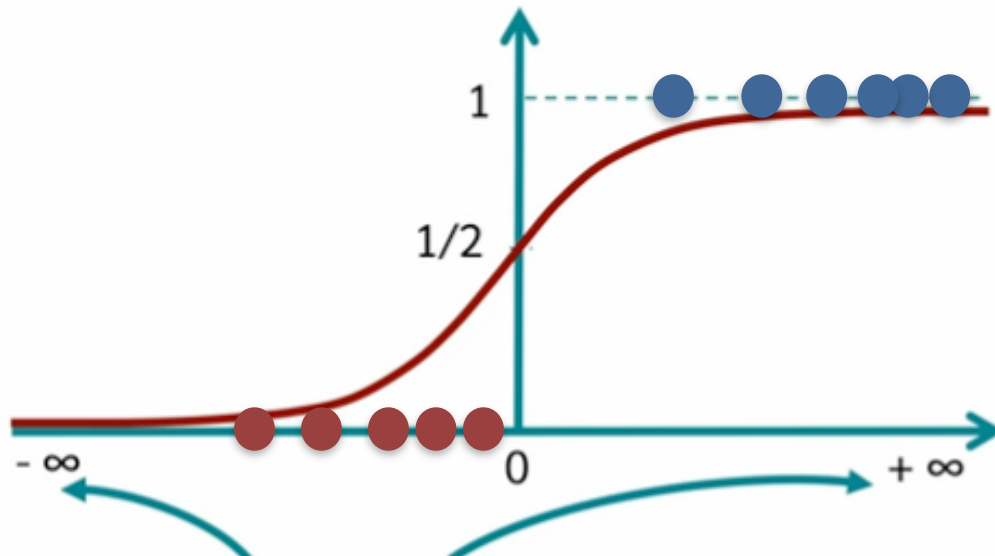
* Por lo tanto, el intervalo de valores de la predicción deberá estar comprendido entre 0 y 1. (*Recordemos que la probabilidad se mide entre 0 y 1*).

* Por eso mismo, necesitamos una función que **solo tome valores entre cero y uno**.



Regresión logística

Eso es exactamente lo que hace la función logística o sigmoide. No importa donde estemos en el eje **X**, ya que entre menos infinito y más infinito, la función siempre nos devolverá valores entre 0 y 1.



Menos infinito,
el valor de y
tiende a cero

Más infinito,
el valor de y tiende
a uno

La **función logística** es la siguiente:

$$f(z) = \frac{1}{1 + e^{-z}}$$

- Salida en (0, 1)
- Monótona creciente
- $p = 0.5$ cuando $z = 0$
- Saturación en los extremos
- Derivable
- Interpretable como probabilidad

Regresión logística

La **función logística** es la siguiente:

$$f(z) = \frac{1}{1 + e^{-z}}$$

Donde **z** va a ser igual a lo siguiente:

$$z = \hat{y} = b_1 \cdot x_1 + b_2 \cdot x_2 + \dots + b_k \cdot x_k + a$$

Resultando en la siguiente ecuación:

$$P(y = 1 \mid X) = \frac{1}{1 + e^{-(b_1 \cdot x_1 + \dots + b_k \cdot x_k + a)}}$$

*“La probabilidad de que la variable dependiente **y** sea igual a 1, dado el conjunto de variables independientes **X**”*

Regresión logística

Ejemplo:

$$P(\text{está enfermo}) = \frac{1}{1 + e^{-(b_1 \cdot \text{edad} + b_2 \cdot \text{sexo} + b_3 \cdot \text{hábito_de_fumar} + a)}}$$

Para poder generar un modelo de ML que permita **determinar si una persona esta enferma dadas sus características X**, tenemos que encontrar los coeficientes “b”.

Este problema de optimización se resuelve con el método de **máxima verosimilitud** o con métodos numéricos.

$$\ell(\beta) = \sum_{i=1}^n \left[y_i \cdot x_i^\top \beta - \log \left(1 + e^{x_i^\top \beta} \right) \right]$$

$$\hat{\beta}_{\text{MLE}} = \arg \max_{\beta \in \mathbb{R}^{k+1}} \ell(\beta)$$

Interpretando los coeficientes:

Para poder generar un modelo de ML que permita **determinar si una persona esta enferma dadas sus características X**, tenemos que encontrar los coeficientes “b”.

$$P(\text{está enfermo}) = \frac{1}{1 + e^{-(b_1 \cdot \text{edad} + b_2 \cdot \text{sexo} + b_3 \cdot \text{hábito_de_fumar} + a)}}$$

Supongamos que en el ejemplo anterior, el coeficiente b_1 de la variable *edad* tiene un valor de 0.04. En ese caso, **el signo positivo nos indica que un aumento en *Edad* va asociado a un incremento en la probabilidad de sufrir la enfermedad.**

* Si todas las variables están **normalizadas** (media 0, varianza 1), podemos comparar probabilidades entre variables.

★ Momios ($p/1-p$) (Odds)

Los momios son la razón de probabilidad de ocurrencia de un evento entre la probabilidad de no ocurrencia de un evento.

$$\text{Momio: } p/(1-p)$$

Donde $p/1-p$ puede tomar valores desde 0 a infinito.

A partir de esta fórmula:

$$P(y = 1 \mid X) = \frac{1}{1 + e^{-(b_1 \cdot x_1 + \dots + b_k \cdot x_k + a)}}$$

Sus momios se calculan de esta manera:

$$\frac{p}{1-p} = e^{a+b_1x_1+\dots+b_kx_k}$$

Y los *log-momios* se obtienen con la siguiente fórmula:

$$\log \left(\frac{p}{1-p} \right) = \log(e^z) = z = a + b_1 x_1 + \cdots + b_k x_k$$

En resumen:

$$\underbrace{p = \frac{1}{1 + e^{-z}}}_{\text{sigmoide}} \iff \underbrace{\frac{p}{1-p} = e^z}_{\text{momios}} \iff \underbrace{\log \frac{p}{1-p} = z}_{\text{logit}}$$

Ejemplo:

Probabilidad de tener empleo formal (`formal = 1`) en función de los años de escolaridad y el sexo:

$$\log \left(\frac{p}{1-p} \right) = \alpha + \beta_1 \cdot \text{escolaridad} + \beta_2 \cdot \text{mujer}$$

Supón que estimas con R y te dan los siguientes coeficientes:

Parámetro	Estimación
α (intercepto)	-2.0
β_1 (escolaridad, años)	0.25
β_2 (mujer = 1)	-0.35

$\beta_1 = 0.25$ significa que un año adicional de escolaridad aumenta los **log-momios** (logaritmo natural de la razón de momios) de tener empleo formal en 0.25 unidades, *ceteris paribus*.

Si **exponenciamos** el log de los momios:

$$e^{\beta_1} = e^{0.25} \approx 1.284$$

Ejemplo:

* Si **exponenciamos** el log de los momios:

$$e^{\beta_1} = e^{0.25} \approx 1.284$$

Cada año adicional de escolaridad **multiplica los momios** de empleo formal por **1.284**, manteniendo el sexo constante.

* Si obtenemos la **tasa de cambio**:

$$(e^{\beta} - 1) \times 100$$

Escolaridad: $(1.284 - 1) \times 100 = + 28.4\%$ por año (aumento en los momios).

Ejemplo:

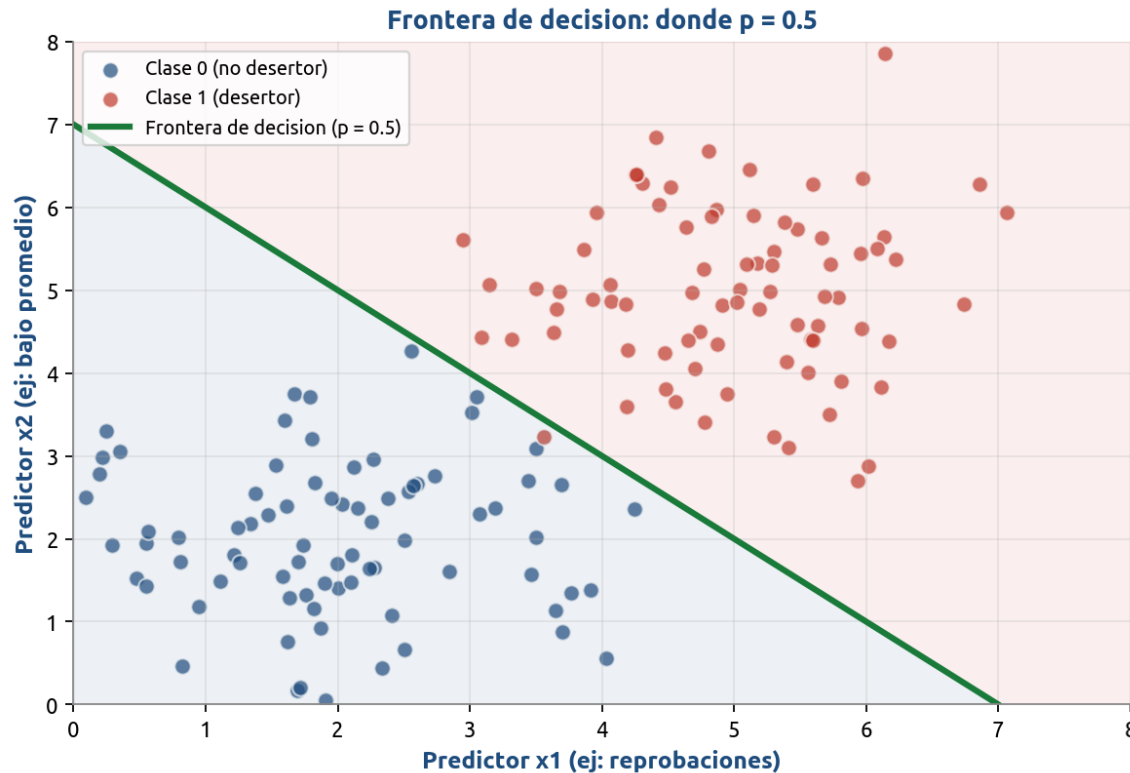
* Probabilidades predichas (entre 0 y 1), **reemplazando valores:**

$$p = \frac{1}{1 + e^{-(\alpha + \beta_1 X_1 + \beta_2 X_2)}}$$

Algunos perfiles concretos:

Perfil	Escolaridad	Mujer	Logit	p
Hombre, primaria completa	6	0	-0.50	0.378
Hombre, prepa completa	12	0	1.00	0.731
Hombre, universidad	16	0	2.00	0.881
Mujer, primaria completa	6	1	-0.85	0.299
Mujer, prepa completa	12	1	0.65	0.657
Mujer, universidad	16	1	1.65	0.839

Frontera de decisión



¿Que es?

El conjunto de puntos en el espacio de predictores donde el modelo asigna probabilidad 0.5 a cada clase.

En regresión logística

La frontera es siempre LINEAL. Por eso a la regresión logística se le llama clasificador lineal: separa con una recta (en 2D) o un hiperplano (en mas dimensiones).

Regresión logística con *tidymodels*

El proceso es muy similar.

La principal diferencia está en la definición del modelo:

Definimos el modelo

```
logistic_model <- logistic_reg() %>%  
  set_engine('glm') %>%  
  set_mode('classification')
```

Engine: 'glm'

Modo: clasificación

Y en las medidas de evaluación de los modelos de clasificación: *Accuracy*, *specificity*, *sensibility*, *curva ROC*, *AUC*.

Caso de estudio - deserción escolar

Predicción temprana de deserción estudiantil

Contexto

Una universidad quiere identificar de forma temprana a los estudiantes con **MAYOR PROBABILIDAD** de abandonar sus estudios durante el primer año, para canalizarlos a programas de tutoría y acompañamiento académico.

Variable resultado

deserto: factor con dos niveles

- "si" (clase positiva)
- "no"

Importante: "si" se define como el PRIMER nivel del factor para que tidymodels la trate como **positiva**.

Por que importa

- **Costo humano:** estudiante desertor pierde acceso académico
- **Costo institucional:** retención y reputación
- **Costo del falso negativo es alto:** dejar fuera a alguien que SI necesitaba apoyo

El dataset

1,000 estudiantes con 14 predictores

Numéricos académicos

- promedio_prepa
- examen_admision
- horas_estudio_semanal
- horas_lectura_semanal
- asistencia_pct
- reprobaciones

Numéricos socioeconomicos

- edad
- ingreso_familiar_miles
- distancia_campus_km

Catégoricos

- carrera (5 niveles)
- tipo_beca (3 niveles)
- trabaja (si / no)
- vive_con_familia (si / no)
- inscrito_sistema (si / no)

Problemas insertados a proposito

- NAs en 4 columnas (3-8%)
- Colinealidad: horas_lectura ~ horas_estudio
- Var. casi nula: inscrito_sistema (99% "si")
- Escalas heterogeneas: miles vs porcentajes

Pipeline tidymodels para clasificación

Mismos pasos que regresión, con detalles específicos

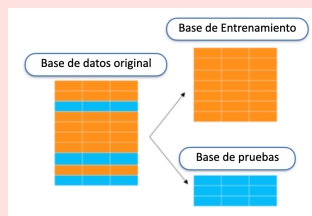
1. Cargamos tidymodels y cargamos los datos

Cargamos tidyverse, tidymodels
set.seed para reproducibilidad.



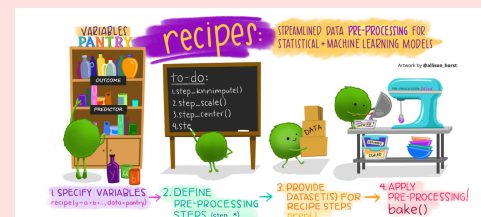
2. Dividimos en base de datos de entrenamiento y de prueba

initial_split(strata = deserto)
Mantiene la proporción de clases



3. Definir receta

Creamos la receta, le añadimos los pasos, la entrenamos y generamos las bases horneadas



4. Especificar modelo

```
logistic_reg() %>%  
set_engine("glm") %>%  
set_mode("classification")
```

5. Ajustar y predecir

Ajustamos el modelo sobre la base de
entrenamiento con *fit()* sobre la base
de entrenamiento "horneada" y luego
predecimos los valores de acuerdo al
modelo y a los valores de **X**.
predict(type = "class" y type = "prob")

6. Evaluar el desempeño

Con los datos reales y los predichos,
evaluamos el desempeño del modelo
(ROC, AUC, spec, accuracy)

1) Cargamos los datos y las librerías

```
# -----  
# 1. Carga de paquetes  
# -----  
library(tidyverse)  
library(tidymodels)  
  
set.seed(2026)  
  
estudiantes <- readxl::read_xlsx("estudiantes.xlsx")
```

2) Hacemos la partición estratificada

```
estudiantes_split <- initial_split(estudiantes, prop = 0.75,  
                                   strata = deserto)  
  
estudiantes_training <- estudiantes_split %>% training()  
estudiantes_test      <- estudiantes_split %>% testing()  
  
print(paste("Filas en entrenamiento:", nrow(estudiantes_training)))  
print(paste("Filas en prueba:",        nrow(estudiantes_test)))  
  
# Confirmamos que las proporciones se mantienen  
estudiantes_training %>% count(deserto) %>% mutate(prop = n / sum(n))  
estudiantes_test      %>% count(deserto) %>% mutate(prop = n / sum(n))
```


3) Recipes: orden recomendado

El orden importa: cada paso depende del anterior

1

Imputar

```
step_impute_median()  
step_impute_mode()
```

Los pasos siguientes no toleran NAs

2

Var. casi nula

```
step_nzv()
```

Quita variables sin informacion (ej: 99% "si")

3

Colinealidad

```
step_corr(threshold = 0.85)
```

Filtra predictores muy correlacionados

4

Normalizar

```
step_normalize()
```

Z-score: media 0, sd 1. Necesario antes de dummies

5

Dummies

```
step_dummy()
```

Convierte factores en variables binarias

3) Aplicamos la receta

Pasos 1 y 2 de la receta

```
receta_estudiantes <- recipe(deserto ~ ., data = estudiantes_training) %>%  
  # 1. Imputación  
  step_impute_median(all_numeric_predictors()) %>%  
  step_impute_mode(all_nominal_predictors()) %>%  
  # 2. Varianza casi nula  
  step_nzv(all_predictors()) %>%  
  # 3. Colinealidad  
  step_corr(all_numeric_predictors(), threshold = 0.85) %>%  
  # 4. Normalización z-score  
  step_normalize(all_numeric_predictors()) %>%  
  # 5. Dummies  
  step_dummy(all_nominal_predictors())  
  
receta_estudiantes
```

3) Aplicamos la receta

Paso 3 de la receta:

```
# 5.1 Entrenamiento de la receta
# prep() aprende las medianas, medias, desviaciones y umbrales únicamente
# sobre los datos de entrenamiento. Esto evita fugas de información (data
# leakage).
receta_estudiantes_prep <- receta_estudiantes %>%
  prep(training = estudiantes_training)

receta_estudiantes_prep

# 5.2 Aplicación a entrenamiento y prueba
estudiantes_training_prep <- receta_estudiantes_prep %>%
  bake(new_data = NULL)

estudiantes_test_prep <- receta_estudiantes_prep %>%
  bake(new_data = estudiantes_test)

glimpse(estudiantes_training_prep)
```

Especificación y entrenamiento

```
modelo_logistico <- logistic_reg() %>%  
  set_engine("glm") %>%  
  set_mode("classification")  
  
logistic_fit <- modelo_logistico %>%  
  fit(deserto ~ ., data = estudiantes_training_prep)  
  
logistic_fit
```

```
# 6.1 Coeficientes del modelo, ordenados por significancia  
# Coeficientes positivos aumentan la probabilidad de pertenecer a la clase  
# positiva (si deserto), negativos la disminuyen.  
tidy(logistic_fit) %>%  
  arrange(p.value)
```

parsnip estandariza la sintaxis: el mismo código serviría para cambiar a un random forest cambiando solo la especificación del modelo.

Predicción de etiquetas y probabilidades

```
# -----  
# 7. Predicción sobre el conjunto de prueba  
# -----  
# Para clasificación necesitamos dos tipos de predicción:  
# - type = "class": la clase predicha (si o no).  
# - type = "prob": las probabilidades estimadas para cada clase  
# (necesarias para la curva ROC).
```

```
class_preds <- predict(logistic_fit,  
                      new_data = estudiantes_test_prep,  
                      type = "class")
```

```
prob_preds <- predict(logistic_fit,  
                    new_data = estudiantes_test_prep,  
                    type = "prob")
```

```
# Combinamos resultados  
resultados_test <- estudiantes_test_prep %>%  
  select(deserto) %>%  
  bind_cols(class_preds, prob_preds)
```

```
resultados_test
```

type = "class"

La etiqueta final, ya aplicado el threshold por defecto (0.5).

type = "prob"

Las probabilidades para cada clase. Indispensables para ROC, AUC y ajustar el threshold.

Interpretación de coeficientes del modelo

Que nos dicen los betas estimados

Recordatorio: variables ya normalizadas

Los coeficientes estan en escala z-score, por lo que SI son comparables entre si.
El de mayor magnitud absoluta es el predictor mas influyente.

Aumentan riesgo ($b > 0$)

- reprobaciones
- trabaja_si
- vive_con_familia_no

Coeficiente positivo en el LOGIT -> mayor probabilidad de pertenecer a la clase "si" (desertor).

Reducen riesgo ($b < 0$)

- promedio_prepa
- asistencia_pct
- horas_estudio_semanal
- tipo_beca_completa

Coeficiente negativo -> menor probabilidad de desertar.

Evaluación del modelo

```
# -----  
# 8. Evaluación del modelo  
# -----  
  
# 8.1 Matriz de confusión  
conf_mat(resultados_test,  
          truth      = deserto,  
          estimate = .pred_class)  
  
# Visualización tipo heatmap  
conf_mat(resultados_test,  
          truth      = deserto,  
          estimate = .pred_class) %>%  
  autoplot(type = "heatmap") +  
  labs(title = "Matriz de confusión")
```

Matriz de confusion

Cuatro escenarios en una sola tabla

Matriz de confusion

		Realidad	
		Positivo	Negativo
Prediccion	Positivo	VP Verdadero Positivo	FN Falso Negativo
	Negativo	FP Falso Positivo	VN Verdadero Negativo

Verdaderos

VP: el modelo dijo "si" y era "si".

VN: el modelo dijo "no" y era "no".

Son los aciertos.

Errores

FP (error tipo I): el modelo dijo "si" pero era "no". En temas de inocencia, gasto de recursos o contratación, este es el error más costoso.

FN (error tipo II): el modelo dijo "no" pero era "si". En desertión escolar o detección de enfermedades mortales, este es el error mas costoso.

Métricas básicas

Tres preguntas, tres respuestas

Accuracy

$(VP + VN) / \text{Total}$

¿Que porcentaje de las predicciones acertó el modelo?

Cuidado: engañosa con clases desbalanceadas. Un modelo que siempre dice "no" tendría 85% si solo 15% deserta.

Sensibilidad

$VP / (VP + FN)$

De los que SI desertaron, ¿cuantos detecto el modelo?.

También llamada RECALL o TPR.
Critica cuando perder un positivo es costoso (desercion, cancer, fraude).

Especificidad

$VN / (VN + FP)$

De los que NO desertaron, ¿cuantos clasifico bien?.

Mide la "limpieza" del filtro: que tan bien evita alarmas innecesarias.

Precision, recall, F1

Cuando importa la calidad de las alarmas

Precision

$$VP / (VP + FP)$$

De todos los que el modelo MARCO como desertores, que porcentaje realmente desertaron.

Recall (= Sensibilidad)

$$VP / (VP + FN)$$

De todos los desertores REALES, cuantos detecto el modelo.

¿Cuándo usar cual?

- **Alta precisión:** cuando el costo del FALSO POSITIVO es alto (ej: cancelar credito por error).
- **Alto recall:** cuando el costo del FALSO NEGATIVO es alto (ej: no detectar a un estudiante en riesgo).

Evaluación del modelo

```
# 8.2 Métricas individuales
# Sensibilidad: de todos los estudiantes que efectivamente desertaron, qué
# porcentaje detectó el modelo (queremos que sea alta para no dejar fuera a
# quien necesita apoyo).
# Especificidad: de todos los estudiantes que NO desertaron, qué porcentaje el
# modelo clasificó correctamente.
accuracy(resultados_test, truth = deserto, estimate = .pred_class)
sens(resultados_test,      truth = deserto, estimate = .pred_class)
```

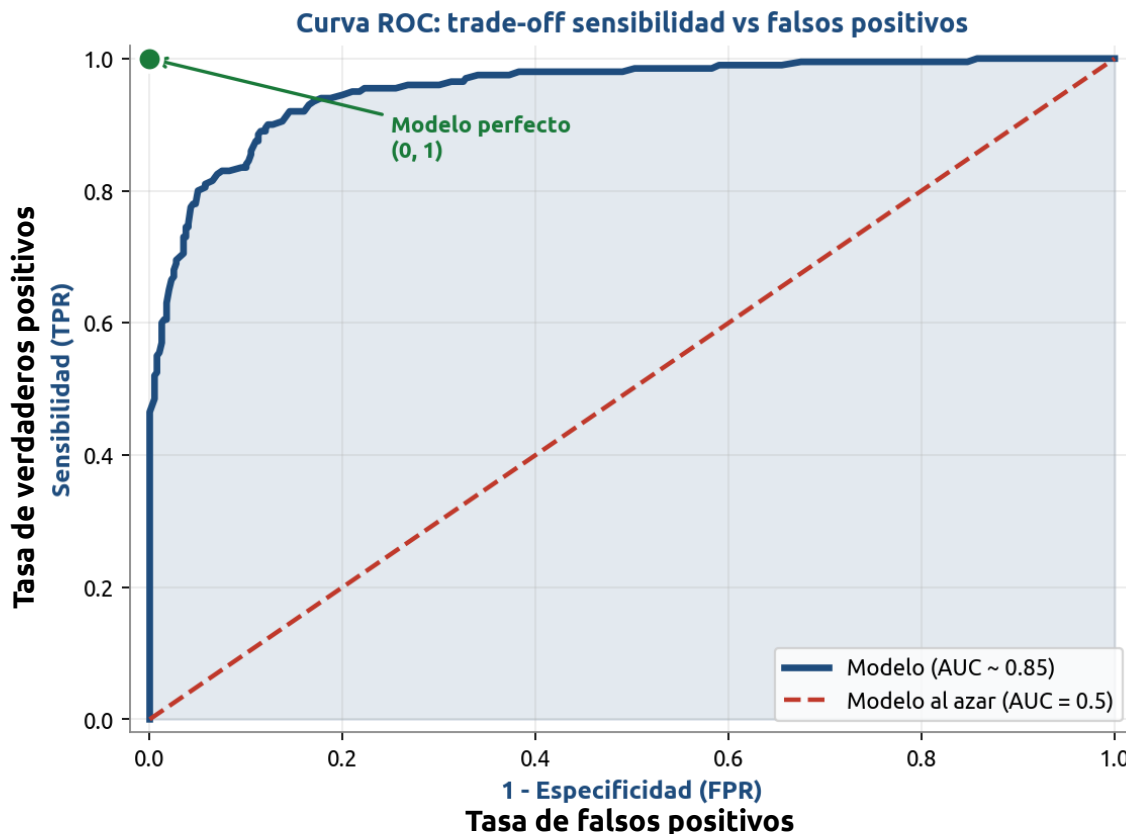
```
# 8.3 Conjunto de métricas personalizado
metricas_clasificacion <- metric_set(accuracy, sens, yardstick::spec)

metricas_clasificacion(resultados_test,
                        truth      = deserto,
                        estimate   = .pred_class)

# 8.4 Resumen completo desde la matriz de confusión
conf_mat(resultados_test,
          truth      = deserto,
          estimate   = .pred_class) %>%
summary()
```

Curva ROC

Receiver Operating Characteristic, nombre heredado del radar de la Segunda Guerra Mundial. Es una herramienta gráfica para evaluar clasificadores binarios. Se grafica: Sensibilidad vs (1 - especificidad) para todos los umbrales de decisión.



¿Que muestra?

Como cambian sensibilidad y especificidad al mover el threshold. La esquina superior izquierda es el ideal (sens = 1, FPR = 0).

Cómo leerla

- **Esquina superior izquierda (0,1):** el clasificador perfecto — detecta a todos los positivos sin falsos positivos.
- **Diagonal:** clasificador aleatorio. Una curva pegada a la diagonal no aporta información más allá del azar.
- **Curva que se aleja hacia arriba-izquierda:** mejor desempeño.

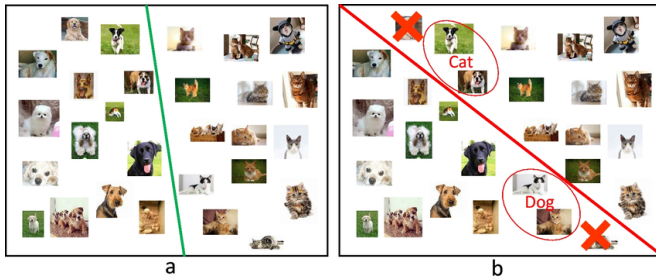
La intuición

Cualquier clasificador probabilístico (logística, random forest, etc.) entrega una probabilidad $p \in [0,1]$. Para convertirla en una clase necesitas un **umbral** (típicamente 0.5, pero es arbitrario).

La curva ROC muestra qué pasa con tu clasificador conforme mueves ese umbral de 1 hacia 0.

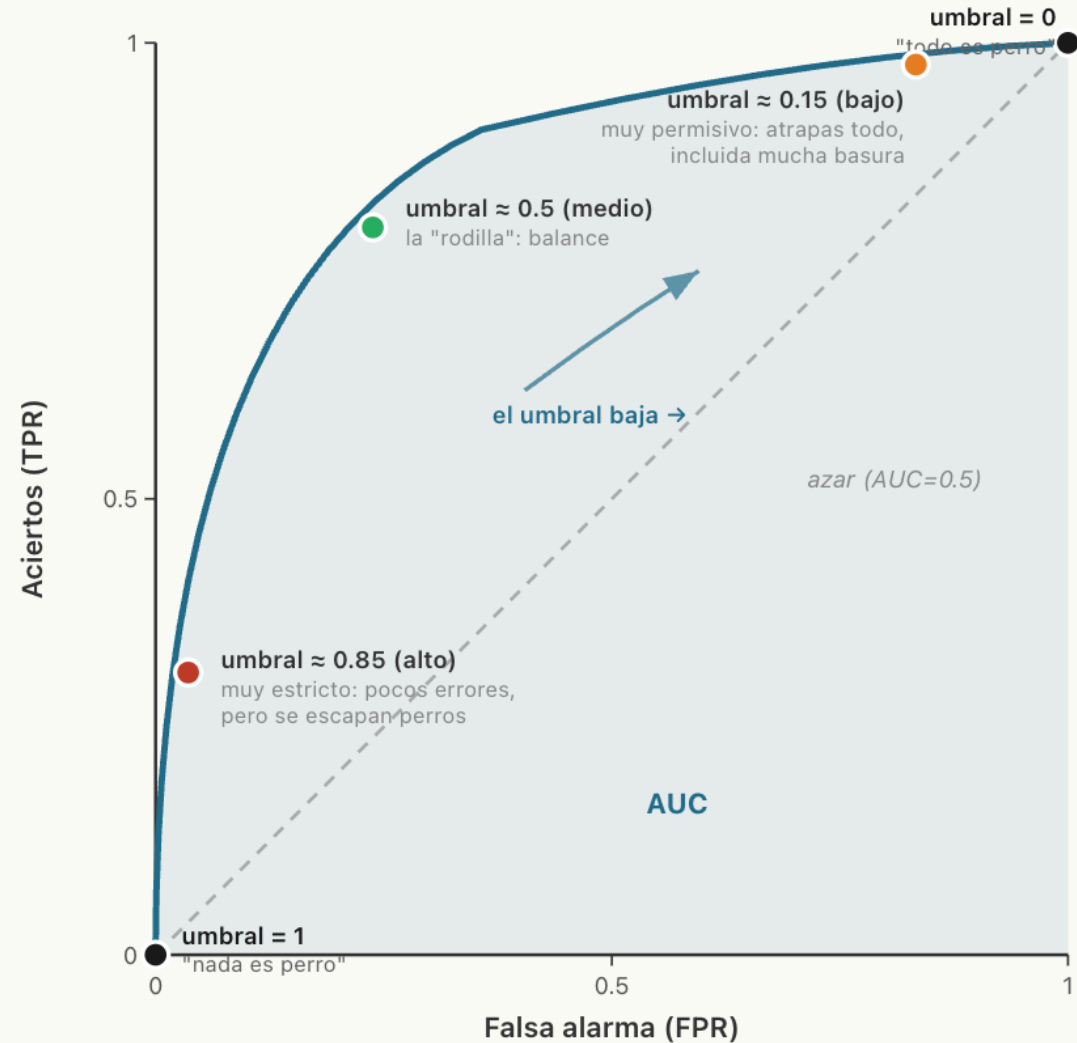
Curva ROC

Imagina que estás programando un modelo para detectar si una imagen es de perros o de no-perros.

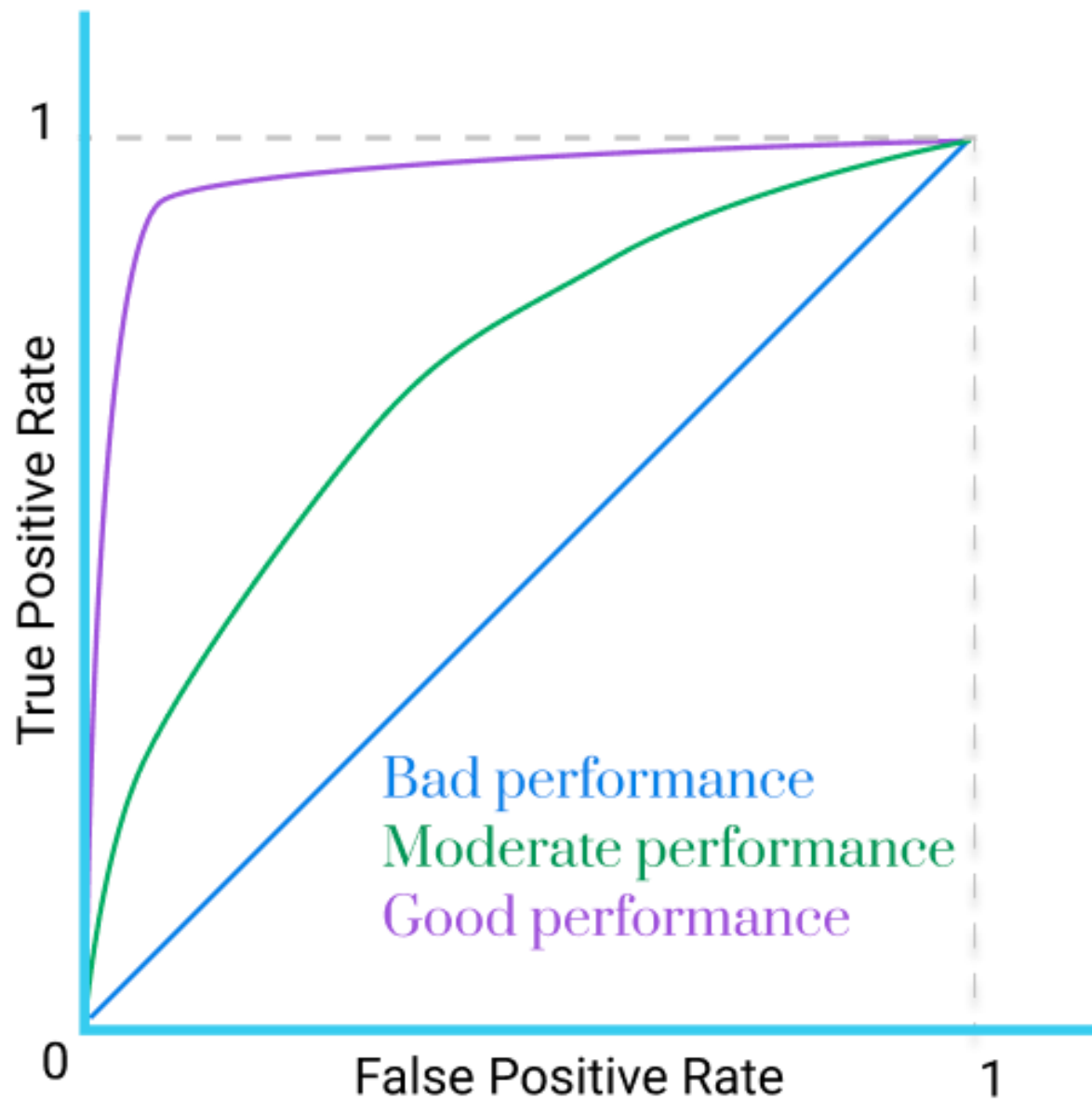


Curva ROC

El umbral no está en los ejes: vive a lo largo de la curva



Curva ROC



Curva ROC y AUC

```
# 8.5 Curva ROC y AUC
# Calculamos los puntos de la curva
roc_df <- resultados_test %>%
  roc_curve(truth = deserto, .pred_si)

roc_df

# Graficamos
roc_df %>%
  autoplot() +
  labs(title = "Curva ROC del modelo de deserción")

# Área bajo la curva
# Interpretación de AUC:
#   AUC = 0.5 -> modelo equivalente a azar.
#   AUC ~ 0.7 -> desempeño aceptable.
#   AUC ~ 0.8 -> buen desempeño.
#   AUC > 0.9 -> excelente desempeño.
roc_auc(resultados_test, truth = deserto, .pred_si)
```

AUC: área bajo la curva ROC

Un solo numero que resume el desempeño

AUC en $[0, 1]$: probabilidad de rankear bien un par

Probabilidad de que el modelo asigne mayor score a un positivo aleatorio que a un negativo aleatorio

AUC = 0.5 - Modelo equivalente al azar

AUC ~ 0.7 - Desempeno aceptable

AUC ~ 0.8 - Buen desempeno

AUC > 0.9 - Excelente desempeno

```
roc_auc(resultados_test, truth = deserto, .pred_si)
```


Ajustar el threshold de decision

Cuando el default de 0.5 no es lo optimo

```
# Threshold por defecto: 0.5
# Cambiar a 0.3 para favorecer sensibilidad
resultados_03 <- resultados_test %>%
  mutate(.pred_class_03 = factor(
    if_else(.pred_si > 0.3, "si", "no"),
    levels = c("si", "no")
  ))

conf_mat(resultados_03,
  truth = deserto, estimate = .pred_class_03) %>%
  summary()
```

Threshold mas bajo (0.3)

Mas alarmas, mayor sensibilidad. Util cuando perder un positivo es muy costoso.

Threshold mas alto (0.7)

Menos alarmas, mayor precision. Util cuando un falso positivo es caro y queremos solo alarmas seguras.

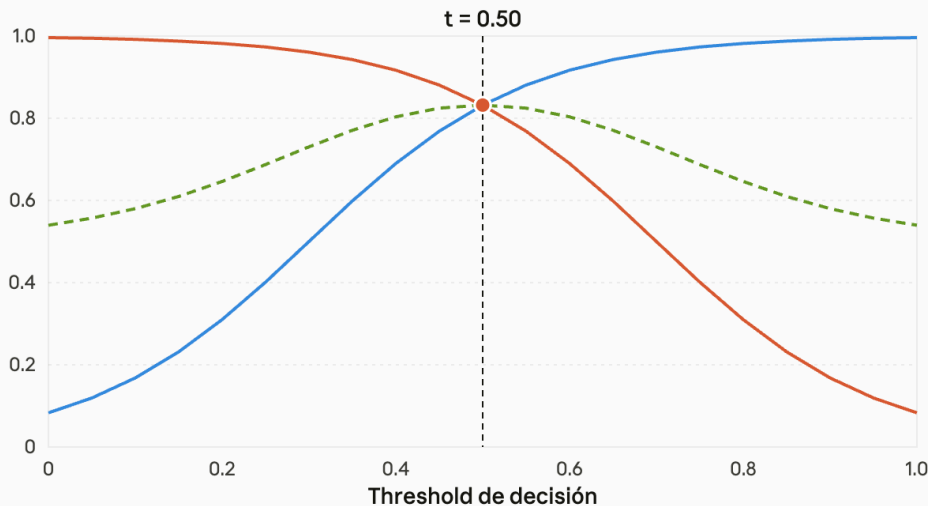
Threshold: el umbral de decisión

Default

0.5 - clasificamos como clase positiva si $p > 0.5$

Ajustar según costo

Si no detectar un positivo es muy costoso (ej: cancer, desercion), bajar el threshold a menos de 0.5 (como 0.3) aumenta la sensibilidad a costa de mas falsos positivos.



Mueve el umbral

0.50

■ Sensibilidad

0.83

■ Especificidad

0.83

■ Accuracy

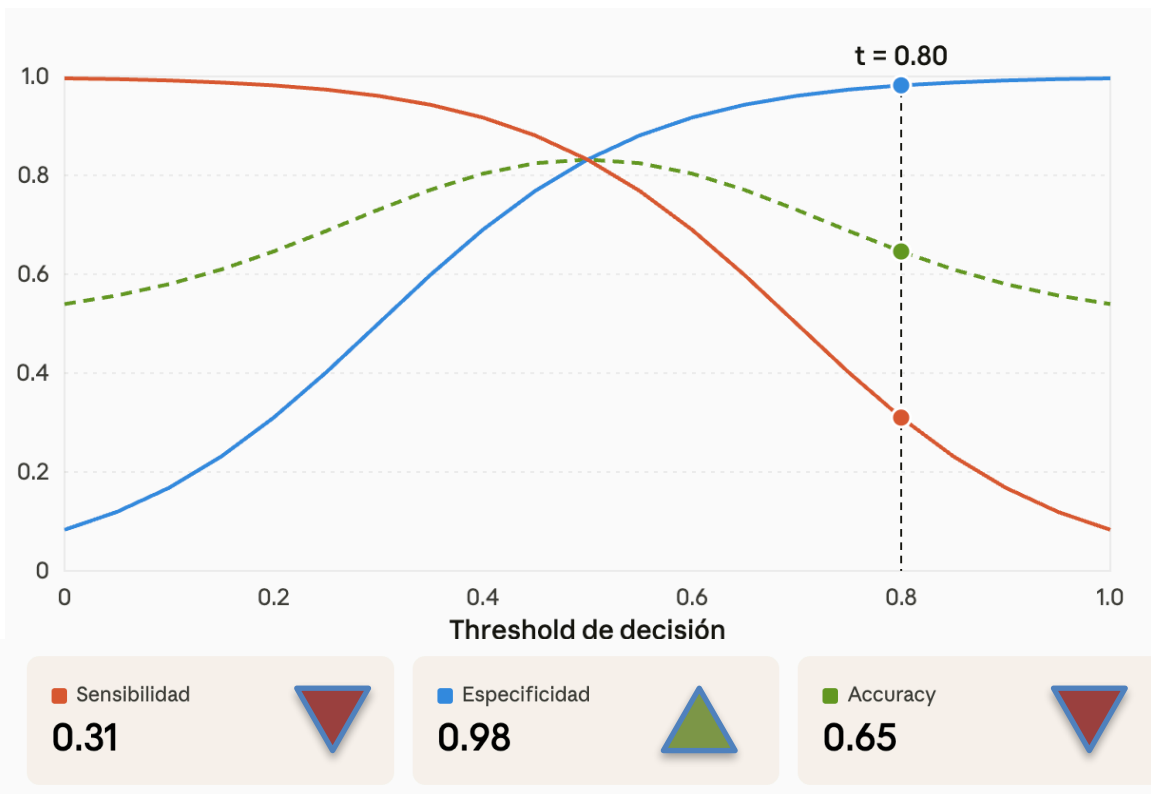
0.83

De los que SI desertaron,
¿cuantos detecto el modelo?.

— Sensibilidad (detectar positivos)
— Especificidad (rechazar negativos)
— Accuracy

Threshold = 0.5: la convención *por defecto* de los paquetes de ML. No es una elección óptima en sí — es solo el punto donde el modelo cree que la probabilidad supera el azar.

Threshold: el umbral de decisión



— Sensibilidad (detectar positivos)
— Especificidad (rechazar negativos)
- - Accuracy

Default

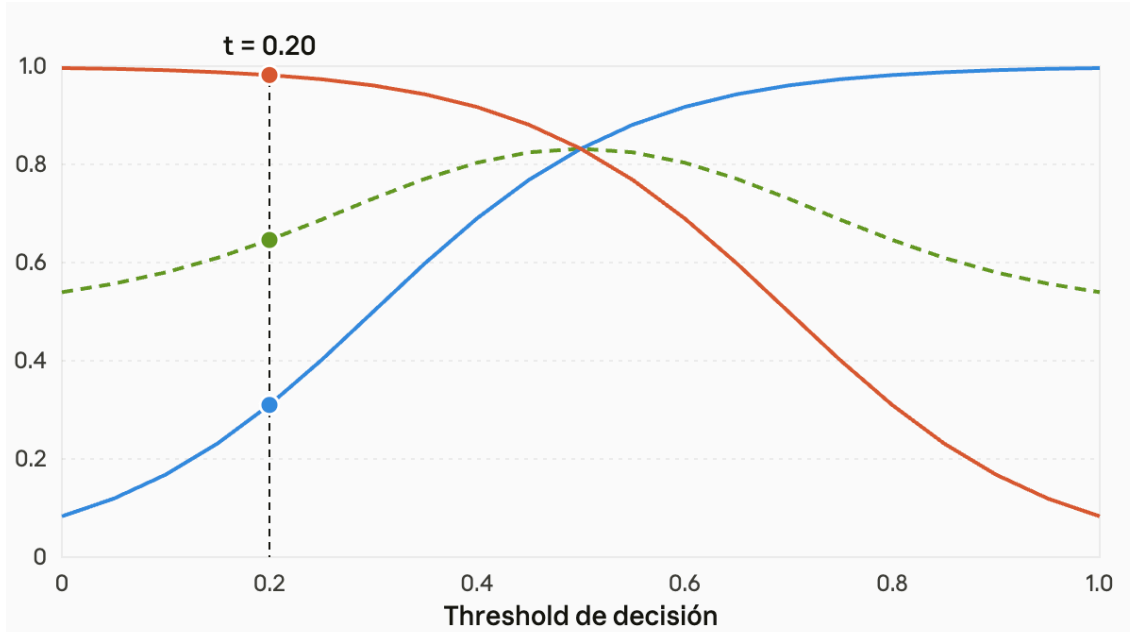
0.5 - clasificamos como clase positiva si $p > 0.5$

Ajustar según costo

Si no detectar un positivo es muy costoso (ej: cancer, desercion), bajar el threshold a menos de 0.5 (como 0.3) aumenta la sensibilidad a costa de mas falsos positivos.

Threshold alto ($t = 0.8$): clasificador "exigente". **Solo etiqueta como positivo cuando está muy seguro.** Especificidad altísima, pero sensibilidad cae (te pierdes muchos positivos reales). Útil cuando un falso positivo es muy costoso.

Threshold: el umbral de decision



■ Sensibilidad
0.98



■ Especificidad
0.31



■ Accuracy
0.65



— Sensibilidad (detectar positivos)
— Especificidad (rechazar negativos)
— Accuracy

Default

0.5 - clasificamos como clase positiva si $p > 0.5$

Ajustar según costo

Si no detectar un positivo es muy costoso (ej: cancer, desercion), bajar el threshold a menos de 0.5 (como 0.3) aumenta la sensibilidad a costa de mas falsos positivos.

Threshold bajo ($t = 0.2$, por ejemplo): tu clasificador se vuelve "permissivo" — clasifica como positivo casi cualquier cosa. Sensibilidad muy alta (atrapa casi todos los positivos reales) pero especificidad muy baja (muchos falsos positivos). Útil cuando un falso negativo es muy costoso.

Ejercicio práctico

1. Cargue las librerías y los datos de la carpeta *hoteles* para modelar si se canceló o no una reserva de hotel. Inspeccione los datos.
2. Para la variable **y**, *cancelo*, obtenga la proporción de verdaderos y falsos.
3. Obtenga el numero de NAs por columna
4. Remueva el id de la reserva, ya que no es un predictor (¿o sí?)
5. Obtenga la matriz de correlación de las variables numéricas e inspeccione en búsqueda de colinealidad
6. Divida la base en base de entrenamiento y base de prueba, estratificando por la variable "*cancelo*". Verifique si la proporción se mantuvo con la calculada en (2).
7. Genere la receta para aplicar los siguientes pasos de feature engineering: (1) Imputar primero (los pasos siguientes no toleran NAs). (2) Quitar varianza casi nula. (3) Filtrar colinealidad sobre numericos. (4) Normalizar (z-score) ANTES de generar dummies. (5) Generar dummies. Aplique la receta. La fórmula de clasificación sería *cancelo* ~ .
8. Defina el modelo de regresión logística

Ejercicio práctico

9. Ajuste el modelo de regresión logística.
10. Ordene los coeficientes en función de su valor p (p -value). ¿Que significan los coeficientes? ¿Cual es la variable con mayor influencia ante la cancelación?
11. Para cada observación de la base de prueba, obtenga la clase predicha (canceló/no canceló) y la probabilidad de cancelar y no cancelar. Obtenga una tabla que contenga el valor real, el valor predicho, la probabilidad de cancelar y la probabilidad de no cancelar.
12. Obtenga (y grafique) la matriz de confusión del modelo con los datos de prueba
13. Obtenga las métricas de accuracy, sensibility y specificity
14. Obtenga la curva ROC y la medida de AUC. Interprete este valor para dar una calificación cualitativa del modelo.
15. Ajuste el threshold (umbral) de decisión a 0.8. ¿Qué tanto cambia la matriz de confusión?

Workflows - Juntando todo

Supongamos que queremos probar 2 modelos distintos; una regresión logística y un k-vecinos cercanos (knn). En ese caso, añadiríamos un paso adicional:

Paso 1: Declaramos los modelos:

```
# Regresión logística (la que ya tenías)
```

```
modelo_logistico <- logistic_reg() %>%  
  set_engine("glm") %>%  
  set_mode("classification")
```

```
# KNN — neighbors lo dejamos tuneable
```

```
modelo_knn <- nearest_neighbor(  
  neighbors      = tune(),          # hiperparámetro a buscar  
  weight_func    = "rectangular",  
  dist_power     = 2                # distancia euclidiana  
) %>%  
  set_engine("kknn") %>%  
  set_mode("classification")
```

Workflows - Juntando todo

Paso 2. Generamos un objeto *workflow()*

Workflow con el modelo logístico

```
wf_logistico <- workflow() %>%  
  add_recipe(receta_reservas) %>%  
  add_model(modelo_logistico)
```

Workflow con el knn

```
wf_knn <- workflow() %>%  
  add_recipe(receta_reservas) %>%  
  add_model(modelo_knn)
```


Workflows - Juntando todo

Un **workflow** es un objeto contenedor de tidymodels que empaqueta en una sola pieza los componentes de un pipeline de modelado: el preprocesador (una recipe o una fórmula) y el modelo (una especificación de parsnip).

Mecánicamente, el objeto tiene tres "slots":

```
wf <- workflow() %>%  
  add_recipe(receta_reservas) %>% # slot "pre" -> preprocesamiento  
  add_model(modelo_knn)           # slot "fit" -> modelo
```

Workflows - Juntando todo

Mientras el workflow está "vacío" (no se ha llamado `fit()`), guarda **especificaciones**: la receta sin preparar y el modelo sin ajustar.

Cuando llamas `fit(wf, data = training)`, *tidymodels* hace dos cosas en cadena y las guarda dentro del mismo objeto:

- * (1) ejecuta `prep()` sobre la receta usando esos datos, aprendiendo medianas, sd, umbrales de correlación, niveles de los factores, etc., y
- * (2) ejecuta `bake()` para producir la matriz de diseño y ajusta el modelo sobre ella.

El resultado es un workflow "fitted" o ajustado, que contiene **la receta preparada y el modelo entrenado**, listo para `predict()`.

EL WORKFLOW ES LA UNIDAD MÍNIMA DE TRABAJO EN TIDYMODELS, y es la forma en que trabajaremos de ahora en adelante.

Ejercicio práctico

1. Convierta

Gracias

